

WIND TURBINE MAINTENANCE COST FORECAST

PROBLEM STATEMENT

A renewable-energy operator services utility-scale wind turbines across onshore and offshore farms. Before the annual maintenance window is locked, the engineering desk must reserve a maintenance budget for each turbine. Under-budgeting delays corrective work and risks lost generation; over-budgeting ties up capital that could have been used elsewhere in the fleet.

The expected cost is not a straight line. Older turbines and heavily used machines generally cost more to maintain, vibration and prior faults raise the chance of component work, and drive-train design, weather exposure and offshore access change the labour and logistics burden. A forecasting stage is required to learn that shape from turbines already serviced, confirm that the model chosen for the job is the better of the two considered, and issue cost forecasts for turbines entering the next maintenance cycle.

OBJECTIVE

Build a six-stage pipeline that prepares the maintenance exports for modelling, fits two forecasters and stores each of them, ranks the two against turbines held back from fitting, measures the chosen forecaster on those held-back turbines, and applies the stored forecaster to a fresh fleet awaiting budget allocation.

FILE STRUCTURE

```
Project
|-- data
|   |-- wind_turbine_train.csv
|   |-- wind_turbine_test.csv
|   `-- wind_turbine_predict.csv
|-- processed_data
|-- artifacts
|-- output
|-- main.py
`-- requirements.txt
```

DATASET DESCRIPTION

Training Dataset: wind_turbine_train.csv

Column	Role	Description
turbine_id	Record	Reference for the turbine.
inspection_date	Record	Date of the engineering inspection.
wind_farm	Record	Wind farm where the turbine operates.
service_region	Record	Service region responsible for the turbine.
maintenance_vendor	Record	Contracted maintenance vendor.
work_order_ref	Record	Maintenance work-order reference.
turbine_age_months	Feature	Age since commissioning, in months.
operating_hours	Feature	Lifetime operating hours.
rated_capacity_mw	Feature	Rated generating capacity in MW.
prior_fault_count	Feature	Recorded prior faults.
gearbox_vibration_mm_s	Feature	Measured drive-train vibration in mm/s.
drive_train_type	Feature	geared, direct_drive or hybrid.
weather_exposure	Feature	low, moderate, high or extreme.
offshore_installation	Feature	no or yes.
annual_maintenance_cost_lakh	Target	Annual maintenance cost in lakh rupees.

wind_turbine_test.csv holds rows of exactly this shape. wind_turbine_predict.csv holds rows of the same shape with annual_maintenance_cost_lakh absent.

TASK

Implement the following functions and the main block described below inside main.py.

FUNCTION SIGNATURE

```
clean_data(input_path, output_path)
```

REQUIREMENTS

- Drop the six record columns. Keep the eight feature columns and annual_maintenance_cost_lakh where present.
- Remove repeated records, keeping the first occurrence, before measuring any median.
- Fill blanks in operating_hours, rated_capacity_mw and gearbox_vibration_mm_s with the median of their own column.
- Normalise case and stray spaces before encoding.
- Encode drive_train_type: geared 0, direct_drive 1, hybrid 2.
- Encode weather_exposure: low 0, moderate 1, high 2, extreme 3.
- Encode offshore_installation: no 0, yes 1.
- Leave no text column and do not create the target on an export that lacks it.
- Write output_path with no index and return the cleaned DataFrame.

FUNCTION SIGNATURE

```
train_svr_model(processed_path, model_path)
```

REQUIREMENTS

- Fit SVR(kernel="rbf") with default C and epsilon on every training row.
- Separate annual_maintenance_cost_lakh from fitted features.
- Standardise all features with StandardScaler learned from training data only and store scaler and SVR together in a Pipeline.
- Persist the fitted object to model_path and return it.

FUNCTION SIGNATURE

```
train_forest_model(processed_path, model_path)
```

REQUIREMENTS

- Fit RandomForestRegressor(n_estimators=100, max_depth=14, min_samples_leaf=15, random_state=42).
- Separate the target from fitted features.
- Persist to model_path and return the fitted forecaster.

FUNCTION SIGNATURE

```
compare_models(svr_model_path, forest_model_path, features_test, target_test)
```

REQUIREMENTS

- Load both stored forecasters; do not refit.
- Score only the passed held-back rows.
- Report RMSE, MAE and R2. Use root_mean_squared_error from sklearn.metrics.
- Return model_name, rmse, mae, r2_score ordered by r2_score strongest first. Name models SVR and RandomForestRegressor.

FUNCTION SIGNATURE

```
evaluate_model(model_path, features_test, target_test)
```

REQUIREMENTS

- Load the specified model without refitting.
- Return RMSE, MAE and R2 as a tuple in that order.

FUNCTION SIGNATURE

```
predict_new_data(model_path, input_path, processed_path, output_path)
```

REQUIREMENTS

- Load the stored model without refitting.

- Prepare input_path with clean_data and write processed_path.
- Preserve turbine_id from the raw file.
- Write only turbine_id and predicted_annual_maintenance_cost_lakh to output_path with no index.
- Return the prediction DataFrame.

MAIN BLOCK

- Runs all six stages in sequence and is the only place that prints.
- Cleans training and held-back exports, separates held-back features and target, trains and stores both models, ranks them, selects the stored path named in the first comparison row, evaluates it and uses it for prediction.
- Reports cleaned shapes, storage confirmation, comparison table, RMSE/MAE/R2 and prediction shape.

EXPECTED OUTPUT

```
Cleaned training turbines: (27000, 9)
Cleaned held-back turbines: (7715, 9)
Models trained and saved.
Model comparison:
...
RMSE: ... | MAE: ... | R2: ...
Predictions written: (800, 2)
```

IMPLEMENTATION STEPS

```
cd Project
pip install -r requirements.txt
python3 main.py
python3 -m pytest ../pytest/tests.py -v
```